

DSA STUDY BLUEPRINT SERIES

22. Strings and Character Arrays Part1

Null-terminated character arrays and `std::string` objects

Section 04 • C++ Core Data Structures & Algorithms

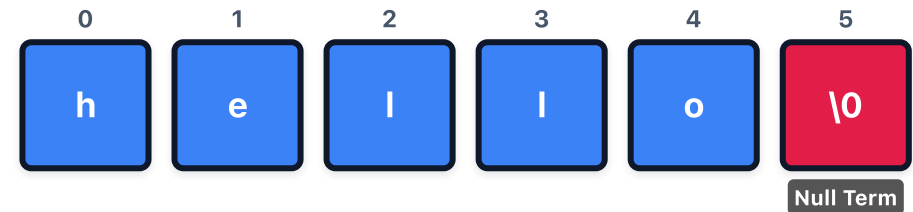
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Char Arrays:** Flat byte structures terminated by `'\0'`.
- **std::string:** Dynamic C++ objects that manage heap buffers automatically.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
char arr[] = "hello"; // null-terminated
string str = "world"; // dynamic string object
string combined = str + " " + arr;
getline(cin, combined); // reads spaces
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Access/Modifications	$O(1)$ / $O(N)$ copies
Space Complexity	$O(N)$

Key Practices & Gotchas

- **Important:** Always verify buffer bounds when writing values to character arrays.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run two-pointer string validations or window updates for string "race" :

Step	Left Pointer Index	Right Pointer Index	Char Comparison	Outcome
1	left = 0 ('r')	right = 3 ('e')	'r' != 'e'	Validation stops (Not a palindrome)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Sliding Window Patterns:** Track character frequency maps within a moving boundary window size to find substring anagrams and permutations.
- **In-place reversals:** Swap characters inside word boundaries using double pointer offsets.
- **Related LeetCode Problems:** #3 Longest Substring Without Repeating Characters, #567 Permutation in String, #344 Reverse String.